

Profiling & Alleviating Performance Bottlenecks in Zeek

Jyotsna Arunkumar

Arnav Batra

Matthew Sickler

The University of Texas at Austin
Austin, USA

ABSTRACT

We present a proposal for identifying and alleviating bottlenecks in Zeek, a network security monitoring system. Prior work has shown that CPU and memory consumption for Zeek, previously called Bro, is subject to the characteristics of the network traffic [2].

We outline a proposal for designing a reproducible profiling framework and using the resulting data to investigate strategies for mitigating bottlenecks. In this project, we will profile Zeek by testing different types of network traffic. Across these experiments, we will measure various performance metrics to identify a bottleneck. We will then design and evaluate a solution in its ability to improve performance.

This project relates to the course as it focuses on network traffic measurement, packet-processing bottlenecks, analysis of various flow types, and load balancing across workers.

1 PROBLEM & MOTIVATION

First developed under the name "Bro" in the 1990s, Zeek is an open-source network security monitoring platform that passively analyzes real-time network traffic, performing actions including protocol parsing, connection tracking, event generation, scripting, and logging. According to the Zeek Project website, it has over 10,000 deployments worldwide [5].

However, Zeek's packet-processing cost can vary substantially with traffic rate, packet size, flow churn, protocol mix, enabled analyzers, and scripts. Within a single Zeek worker, these costs can create processing bottlenecks, which may lead to incomplete visibility into network traffic as packets are dropped from processing queues. In deployments with multiple workers, which are typically assigned packets via flow hashing, this variable cost can also lead to load imbalances across workers. As a result, the actual processing capacity of a worker cluster may be lower than the theoretical maximum, as seen in metrics including throughput and packet drops.

Zeek's users range from network security operators of enterprise networks to academic researchers. Our hope is to mitigate the bottlenecks that prevent them from using the tool and its underlying resources to the fullest potential. By improving on Zeek's packet-processing performance, its users will be able to analyze larger workloads with more accuracy and gain better visibility into network traffic.

2 RELATED WORK

Perhaps the most foundational related work for this project is *Bro: A System for Detecting Network Intruders in Real-Time* [4]. Although Zeek's design has gone through many iterations since this first research paper, it highlights several of the author's priorities for performance, including maximizing throughput and avoiding packet drops. It identifies several techniques for improving performance, including filtering packets in the kernel and avoiding blocking the single thread. It also notes several limiting factors, including the handling of expired timers immediately, even during high load. Furthermore, to simplify the design, they chose to handle packet-related events immediately, a tradeoff that might have contributed to performance bottlenecks.

In *Re-examining the performance bottleneck in a NIDS with detailed profiling*, its authors profile Bro to identify performance bottlenecks and guide future improvements of NIDS [3]. While earlier work largely focused on speeding-up pattern matching, they note that improvements in pattern-matching did not necessarily translate into equivalent improvements in overall system performance, as other processing stages became bottlenecks. In their profiling of Bro, they identify policy script execution as a key limiting factor, leading to their recommendation of greater compilation into "binary codes native to the CPU."

Campbell and Lee take a different approach in *Prototyping a 100G Monitoring System* [1]. They go beyond profiling, attempting to reduce the amount of traffic that Bro analyzes to begin with. Their mitigation identifies large flows, implementing a "shunting" policy to limit packet processing once certain conditions are met. This reduces the load on Bro workers once enough information has been collected, demonstrating a bottleneck mitigation targeted at a particular type of traffic.

3 PROPOSED WORK AND EVALUATION

To begin, we will evaluate Zeek using controlled, reproducible workloads that vary four traffic characteristics: packet rate, packet size, flow churn, and protocol mix. Our assessment will focus on whether these workload characteristics contribute to processing bottlenecks or worker load imbalances. The data we collect may help explain why some flows

are more expensive to process than others and which resources become limiting factors for performance.

Our main metrics will be:

- packet-processing throughput,
- packet-drop rate,
- CPU utilization,
- memory consumption, and
- worker load imbalance (for multi-worker deployments)

For each goal below, we will develop a mitigation for a bottleneck identified during profiling. We will then rerun the same workloads to identify the overhead introduced by the mitigation on the tool's packet processing, using the same metrics above.

3.1 75% Goal

Our first goal will be to profile Zeek and identify a single bottleneck associated with specific traffic characteristics. We will then implement a simple mitigation that addresses the identified bottleneck. For example, if a bottleneck was identified within a single worker, we could attempt to remove unnecessary processing. If a bottleneck regarding worker load imbalance was chosen, we could create a simple modification to Zeek's existing flow-assignment policy to distribute flows more evenly. We will then compare the proposed mitigation against Zeek's existing behavior under the same workload.

Evaluation: This goal first entails the creation of a Zeek benchmarking environment, with reproducible results for various traffic workloads. We should identify a performance bottleneck, then develop and implement a simple mitigation. This mitigation will be considered successful if it demonstrates measurable performance improvements in any of the aforementioned evaluation metrics.

3.2 100% Goal

For this stage of the project, we will design and implement an improved mitigation for the bottleneck identified during profiling, based on the evaluation results of the 75% implementation. Going beyond optimizing for a single motivating workload, our mitigation will strive to be reasonably performant across most traffic workloads.

For example, if a worker load-imbalance is the primary bottleneck, we might implement a cost-aware flow-assignment strategy that considers a new flow's estimated processing cost and the current load of each worker, while still ensuring flow affinity.

Evaluation: While the 75% goal was evaluated according to its performance improvements in one motivating workload, this stage and its improved mitigation will be assessed for improvements across multiple traffic mixes and rates. We will compare it against the unmodified Zeek and the 75% implementation in metrics such as throughput, packet drops,

CPU utilization, memory consumption, and worker load-imbalance, where possible. We will also characterize the overhead introduced by the mitigation and its limitations.

3.3 125% Goal

Our 125% implementation will extend the 100% solution with a more advanced implementation that dynamically analyzes and responds to changes in workload behavior.

If we identify worker imbalance as our chosen bottleneck for system performance, one possible extension is to predict a new flow's processing cost using information from its first few packets (e.g., packet size, header metadata, transport protocol). Another option is to dynamically scale the number of Zeek workers or possibly migrate flows from one worker to another. For other bottlenecks, possible ideas include adapting to changing protocol mixes and/or targeting unusually expensive flows.

Evaluation: This stage will be evaluated by its benefit (and cost) in comparison to the 100% solution and unmodified Zeek implementation. The 125% solution should demonstrate the ability to determine changing workload conditions and adjust accordingly. This solution will be evaluated by the extra performance it provides relative to the cost of the dynamic reasoning.

4 REQUIRED RESOURCES

The proposed work does not require specialized hardware or a multi-GPU cluster.

Our main resources will include:

- **CloudLab** for compute resources
- **tcpreplay** for existing packet captures
- **iperf3** for generating traffic with variable packet sizes and flow patterns with high ease of use
- **DPDK** for generating high-speed traffic and greater control, as needed
- **Linux perf** for profiling CPU usage
- **zeekctl top** for measuring per-worker CPU and memory utilization
- **Zeek's Prometheus tool** for logging per-worker CPU metrics during automated experiments

5 TEAM MEMBER RESPONSIBILITIES

We plan to meet in-person at least once a week. During these meetings, we will set goals for the week and develop designs based on collaborative discussions. We will then complete our assigned work before regrouping. Finally, we plan to check in periodically with Dr. Sengupta to get feedback on our direction and identify missing elements.

The following is an example of how we intend to divide the proposed work. We will each pick one or two of the packet characteristics to profile, collecting the data required

for the remaining parts of the project. Together, we will then identify the bottleneck that appears most promising. For each aforementioned milestone, we will investigate potential mitigation strategies, combine our ideas into a single solution, and implement it together.

REFERENCES

- [1] Scott Campbell and Jason Lee. 2012. Prototyping a 100G Monitoring System. In *2012 20th Euromicro International Conference on Parallel, Distributed and Network-Based Processing*. 293–297. <https://doi.org/10.1109/PDP.2012.95>
- [2] Holger Dreger, Anja Feldmann, Vern Paxson, and Robin Sommer. 2008. Predicting the Resource Consumption of Network Intrusion Detection Systems. In *Recent Advances in Intrusion Detection*. Springer, 135–154. https://doi.org/10.1007/978-3-540-87403-4_8
- [3] Po-Ching Lin and Jia-Hau Lee. 2013. Re-examining the performance bottleneck in a NIDS with detailed profiling. *Journal of Network and Computer Applications* 36 (03 2013), 768–780. <https://doi.org/10.1016/j.jnca.2012.12.009>
- [4] Vern Paxson. 1998. Bro: A System for Detecting Network Intruders in Real-Time. In *7th USENIX Security Symposium (USENIX Security 98)*. USENIX Association, San Antonio, TX. <https://www.usenix.org/conference/7th-usenix-security-symposium/bro-system-detecting-network-intruders-real-time>
- [5] Zeek Project. [n. d.]. *Zeek*. <https://zeek.org/>